

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Task Management Backend API

Detailed project brief generated from the project record.

DURATION

14 days

TECHNOLOGY STACK

Node.js

Express.js

MongoDB

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

Create a RESTful API for users to register, log in, and manage personal tasks.

TYPE

backend

DURATION

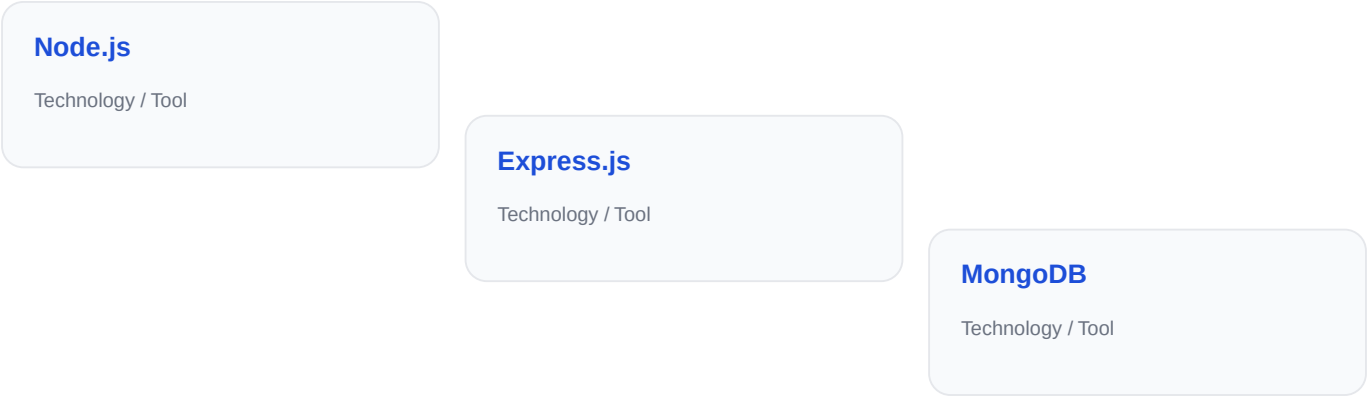
14 days

Primary Objectives

- Implement a secure JWT based authentication flow
- Persist users and tasks in MongoDB
- Provide clear API documentation

Technology Stack

Technologies, frameworks and tools expected for implementation.



Functional Requirements

- Register a new user with email and password
- Log in and receive a JWT
- Create, read, update, delete tasks belonging to the authenticated user
- List tasks with pagination
- Mark tasks as completed
- Search tasks by title keyword

Non-Functional Requirements

- Validate request bodies and query parameters
- Return consistent JSON error responses
- JWT must be signed with a secret
- Response time < 200 ms for CRUD ops
- Swagger/OpenAPI spec for all endpoints

Modules & Features

Major functional areas that make up the project.

Auth

Handles registration, login, and token issuance

- Register endpoint
- Login endpoint

User

Manages user profile data

- Get profile
- Update profile

Task

Core task CRUD and status handling

- Create task
- Update task
- Delete task
- List tasks

Reporting

Provides simple task statistics

- Task count summary
- Completed vs pending stats

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Foundation

Initialize npm project, Add ESLint, Create Express app, Configure Mongoose, Define User and Task schemas, Add .env handling

- Initialize npm project
- Add ESLint
- Create Express app
- Configure Mongoose
- Define User and Task schemas

2

Authentication

Create /register route, Hash passwords with bcrypt, Create /login route, Generate JWT, Add auth middleware, Write unit tests for auth

- Create /register route
- Hash passwords with bcrypt
- Create /login route
- Generate JWT
- Add auth middleware

3

Task CRUD

Create POST /tasks, Create GET /tasks with pagination, Create PATCH /tasks/:id, Create DELETE /tasks/:id, Validate request data, Write integration tests

- Create POST /tasks
- Create GET /tasks with pagination
- Create PATCH /tasks/:id
- Create DELETE /tasks/:id
- Validate request data

4

Reporting

Create GET /tasks/summary, Calculate completed vs pending, Add query filters, Document endpoints, Add tests for summary

- Create GET /tasks/summary
- Calculate completed vs pending
- Add query filters
- Document endpoints
- Add tests for summary

5

Quality & Deployment

Run full test suite, Add coverage report, Write Dockerfile, Create docker-compose.yml, Generate Swagger UI, Prepare release notes

- Run full test suite
- Add coverage report
- Write Dockerfile
- Create docker-compose.yml
- Generate Swagger UI

API Design

Key API endpoints and expected responsibilities.

POST**/register**

Create new user

POST**/login**

Authenticate and return JWT

GET**/tasks**

List user's tasks with pagination

POST**/tasks**

Create a new task

PATCH**/tasks/:id**

Update task fields

DELETE**/tasks/:id**

Delete a task

GET**/tasks/summary**

Return task statistics

Database Design

Recommended data structures and persistence model.

Entity 1

- `_id`
- `email`
- `passwordHash`
- `createdAt`

Entity 2

- `_id`
- `userId`
- `title`
- `description`
- `completed`

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: API correctness | Description: All endpoints meet functional specs | Weight: 30
- Criterion: Code quality | Description: Readable, linted, follows best practices | Weight: 20
- Criterion: Database design | Description: Schemas reflect requirements and use indexes | Weight: 15
- Criterion: Security | Description: Auth, password hashing, input validation | Weight: 15
- Criterion: Testing coverage | Description: Tests cover core flows and edge cases | Weight: 10
- Criterion: Documentation | Description: Swagger spec and README are complete | Weight: 10

Expected Deliverables

Artifacts and outcomes expected at project completion.

- Source code repository
- README with setup and run instructions
- Swagger/OpenAPI documentation
- Dockerfile and docker-compose.yml
- Test suite with coverage report
- Release notes

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

Uses Node/Express and MongoDB skills, matches junior backend role, fits 14 day timeline with clear MVP and incremental quality work.

Additional Project Details

Testing Plan

- Unit tests for auth middleware
- Integration tests for each task endpoint
- Validation tests for request bodies
- Error handling tests for 400/401 responses
- Coverage report "e 80%
- Run tests in CI pipeline

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.